

自计算机诞生以来，许多高价值的应用程序对执行速度和资源的需求超过了计算设备能够提供的范围。早期的应用程序依赖于处理器速度、内存速度和内存容量的进步，以提高应用层面的能力，如天气预报的及时性、工程结构分析的准确性、计算机生成图形的逼真度、每秒处理的航空公司预订数量，以及每秒处理的资金转账数量。

最近，诸如深度学习等新应用程序对执行速度和资源的需求甚至超过了最佳计算设备的能力。这些应用需求推动了过去五十年计算设备能力的快速发展，并将在可预见的未来继续如此。基于单个中央处理单元（CPU）的微处理器，例如英特尔和AMD的386处理器，看似按顺序执行指令，配备快速增加的时钟频率和硬件资源，推动了1980年代和1990年代计算应用性能的快速增长和成本降低。在这两个增长的十年里，这些单CPU微处理器将每秒浮点运算次数（GFLOPS，或十亿次浮点运算）引入了桌面，将每秒浮点运算次数（TFLOPS，或一万亿次浮点运算）引入了数据中心。这种对性能改进的不懈追求使应用软件能够提供更多功能，具有更好的用户界面，并产生更有用的结果。用户反过来一旦习惯了这些改进，就会要求更多的改进，为计算机行业创造了一个积极的（良性）循环。

然而，由于能源消耗和热散热问题，这种推动自2003年以来已经减缓。这些问题限制了时钟频率的提高以及在单个CPU内每个时钟周期内可以执行的生产活动，同时保持顺序执行指令的假象。从那时起，几乎所有微处理器供应商都转向了一种模式，即在每个芯片中使用多个物理CPU，称为处理器核心，以增加处理能力。在这个模型中，传统的CPU可以被看作是单核CPU。为了从多个处理器核心中获益，用户必须拥有多个指令序列，无论是来自相同应用程序还是不同应用程序，这些指令序列可以同时在这些处理器核心上执行。对于特定应用程序要从多个处理器核心中获益，它的工作必须被分成多个指令序列，这些序列可以同时在这些处理器核心上执行。从单个CPU按顺序执行指令切换到多核按并行执行多个指令序列，对软件开发社区产生了巨大的影响。

传统上，绝大多数软件应用程序都是作为顺序程序编写的，这些程序由冯·诺伊曼在他的重要报告中设想的处理器设计执行（冯·诺伊曼等，1972）。这些程序的执行可以被理解为根据程序计数器的概念，也被称为文献中的指令指针，按顺序逐步浏览代码。程序计数器包含处理器将要执行的下一条指令的内存地址。由于这种应用的顺序，逐步执行的指令执行活动序列被称为执行线程，或简称为线程。线程的概念非常重要，它将在本书的其余部分得到更为正式的定义并被广泛使用。

在历史上，大多数软件开发者依赖于硬件的进步，比如时钟速度的增加和在底层执行多个指令，以提高其顺序应用程序的速度；随着每一代新处理器的推出，相同的软件也会变得更快。计算机用户也逐渐期望这些程序在每一代微处理器中都能更快地运行。然而，这一期望在过去十多年来已不再成立。一个顺序程序只能在处理器核心中的一个上运行，而这个核心在每一代中都不会变得显著更快。没有性能提升，应用程序开发者将无法在引入新微处理器时引入新功能和能力，这减少了整个计算机行业的增长机会。

相反，将在每一代新微处理器中继续享有显著性能改进的应用软件将是并程序，其中多个执行线程合作以更快地完成工作。这种并程序相对于顺序程序的这种新的、戏剧性提升的优势被称为并发革命（Sutter和Larus，2005）。并行编程的实践并不新鲜。高性能计算（HPC）社区几十年来一直在开发并程序。这些并程序通常在昂贵的大型计算机上运行。只有少数精英应用程序能够证明使用这些计算机是合理的，因此并行编程的实践一直被限制在少数应用程序开发者中。现在，所有新的微处理器都是并行计算机，需要以并程序的形式开发的应用程序数量急剧增加。现在迫切需要软件开发者学习并行编程，这也是本书的重点。

1.1 异构并行计算

自2003年以来，半导体行业已经确定了设计微处理器的两个主要路径（Hwu等人，2008）。多核路径旨在保持顺序程序的执行速度，同时进入多个核心。多核始于双核处理器，并且随着每一代半导体工艺的进步，核心数量逐渐增加。最近的一个例子是英特尔最新的多核服务器微处理器，最多可达24个处理器核心，每个核心都是乱序、多指令执行处理器，实现完整的386指令集，支持具有两个硬件线程的超线程，旨在最大化顺序程序的执行速度。另一个例子是最近的ARM Ampere多核服务器处理器，具有128个处理器核心。

相比之下，多线程路径更专注于并行应用程序的执行吞吐量。多线程路径始于大量线程，而线程数量再次随每一代增加。最近的一个典型例子是NVIDIA Tesla A100图形处理单元（GPU），具有数以万计的线程，在大量简单、按顺序的流水线中执行。多线程处理器，特别是GPU，自2003年以来一直领先于浮点性能竞赛。截至2021年，A100 GPU的峰值浮点吞吐量为64位双精度为9.7 TFLOPS，32位单精度为156 TFLOPS，16位半精度为312 TFLOPS。相比之下，最近英特尔24核处理器的峰值浮点吞吐量为双精度0.33 TLOPS，单精度0.66 TFLOPS。多线程GPU和多核CPU之间的峰值浮点计算吞吐量比例在过去几年里一直在增加。这些并不一定是应用速度；它们仅仅是这些芯片中执行资源可能支持的原始速度。

多核和多线程之间峰值性能存在如此大的差距，已经积累了显著的“电势”，在某个时刻，必定会有所妥协。我们已经到达了这一点。迄今为止，这种大的峰值性能差距已经激发了许多应用程序开发者将其软件的计算密集部分移至GPU以进行执行。或许更重要的是，并行执行的显著提高性能使得深度学习等革命性新应用程序成为可能，这些应用程序本质上由计算密集的部分组成。毫不奇怪，这些计算密集的部分也是并行编程的主要目标：当有更多的工作要做时，就有更多的机会将工作分配给协作的并行工作者，即线程。

有人可能会问为什么多线程的GPU和多核CPU之间存在如此大的峰值性能差距。答案在于两种处理器类型之间的基本设计理念的差异，如图1.1所示。如图1.1A所示，CPU的设计优化是为了顺序代码性能。算术单元和操作数传递逻辑的设计旨在最小化算术操作的有效延迟，代价是增加单位的芯片面积和功耗。大型的末级片上缓存被设计用来捕获经常访问的数据，并将一些长延迟的内存访问转换为短延迟的缓存访问。先进的分支预测逻辑和执行控制逻辑用于减轻条件分支指令的延迟。通过降低操作的延迟，CPU硬件减少了每个单独线程的执行延迟。然而，低延迟的算术单元、复杂的操作数传递逻辑、大缓存内存和控制逻辑消耗了本可以用于提供更多算术执行单元和内存访问通道的芯片面积和功耗。这种设计方法通常被称为面向延迟设计。

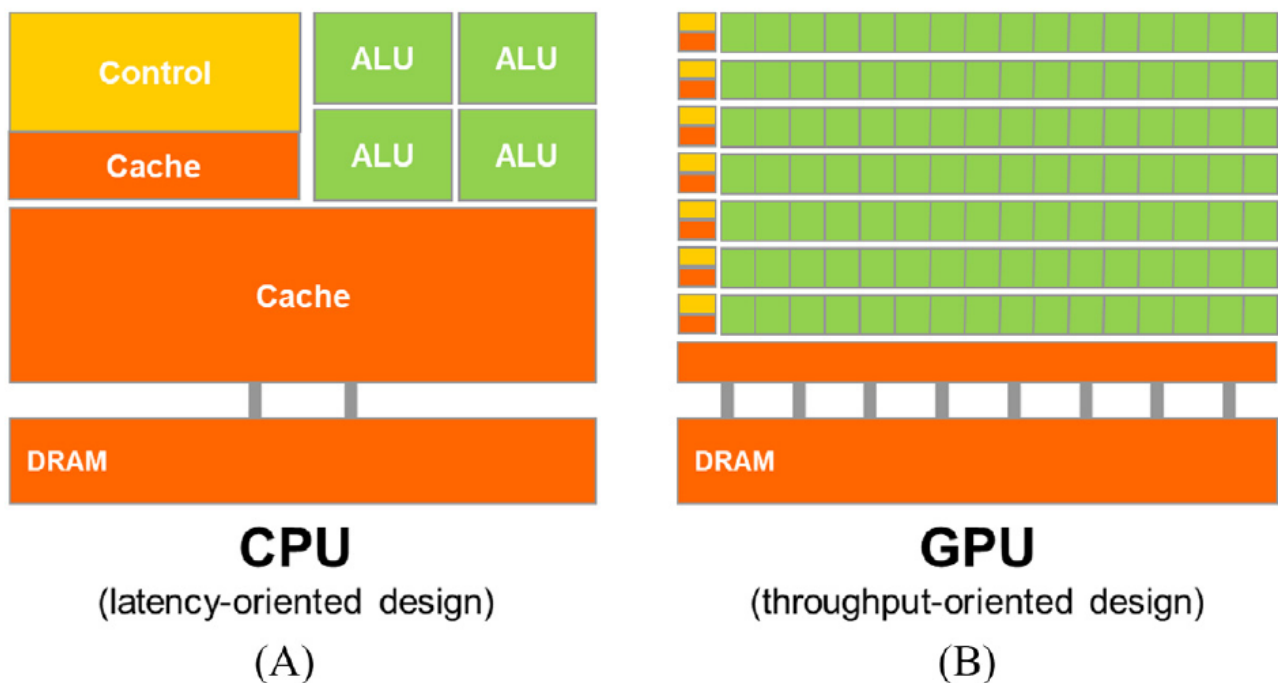


图1.1 CPU和GPU具有根本不同的设计理念：（A）CPU设计是面向延迟的；（B）GPU设计是面向吞吐量的。

另一方面，GPU的设计哲学是由快速增长的视频游戏行业塑造的，该行业对在先进游戏的每个视频帧中执行大量浮点运算和内存访问施加了巨大的经济压力。这种需求激励GPU供应商寻找方法来最大化用于浮点计算和内存访问吞吐量的芯片面积和功耗预算。

在图形应用程序中执行大量浮点运算每秒的需求，例如视点变换和对象渲染的任务，是非常直观的。此外，每秒执行大量内存访问的需求同样重要，甚至可能更为重要。许多图形应用程序的速度受限于数据能够从内存系统传递到处理器以及反之的速率。GPU必须能够将极大量的数据移入和移出其DRAM（动态随机访问存储器）

中的图形帧缓冲区，因为这种移动是使视频显示丰富而令玩家满意的原因。游戏应用程序通常接受的放松的内存模型（系统软件、应用程序和I/O设备对其内存访问方式的期望）也使GPU更容易支持在访问内存时的大规模并行性。

相比之下，通用处理器必须满足来自传统操作系统、应用程序和I/O设备的要求，这些要求对支持并行内存访问提出了更多挑战，因此增加内存访问吞吐量，通常称为内存带宽，变得更加困难。因此，图形芯片的内存带宽通常是同期可用CPU芯片的约10倍，我们预计在一段时间内GPU将在内存带宽方面继续保持优势。

一个重要的观察是，在功耗和芯片面积方面，减少延迟比增加吞吐量昂贵得多。例如，可以通过将算术单元数量翻倍来使算术吞吐量翻倍，代价是将芯片面积和功耗翻倍。然而，将算术延迟减半可能需要将电流翻倍，代价是使用的芯片面积增加超过两倍，并将功耗增加四倍。因此，GPU中的主流解决方案是优化大量线程的执行吞吐量，而不是减少单个线程的延迟。这种设计方法通过允许流水线式内存通道和算术操作具有长延迟来节省芯片面积和功耗。内存访问硬件和算术单元的面积和功耗的降低允许GPU设计者在芯片上拥有更多这些组件，从而增加总执行吞吐量。图1.1通过图示在图1.1A的CPU设计中较少数量的较大算术单元和较少数量的内存通道，与图1.1B中较多数量的较小算术单元和较多数量的内存通道之间的设计方法差异，直观地说明了这种差异。

这些GPU的应用软件预计将以大量并行线程编写。硬件利用大量线程的优势，在其中一些线程等待长延迟内存访问或算术操作时寻找工作。图1.1B中的小缓存存储器旨在帮助控制这些应用程序的带宽要求，以便访问相同内存数据的多个线程不必全部访问DRAM。这种设计风格通常称为面向吞吐量设计，因为它致力于最大化大量线程的总执行吞吐量，同时允许单个线程可能需要更长时间来执行。

很明显，GPU被设计为并行、面向吞吐量的计算引擎，它们在一些CPU设计得很好的任务上性能不佳。对于只有一个或很少线程的程序，具有较低操作延迟的CPU可以实现比GPU高得多的性能。当程序具有大量线程时，具有更高执行吞吐量的GPU可以实现比CPU高得多的性能。因此，人们应该期望许多应用程序同时使用CPU和GPU，将顺序部分在CPU上执行，将数值密集型部分在GPU上执行。这就是为什么NVIDIA在2007年推出的Compute Unified Device Architecture (CUDA) 编程模型被设计为支持应用程序的联合CPU-GPU执行的原因。还要注意的，在应用程序开发者选择运行其应用程序的处理器时，速度并不是唯一的决策因素。还有其他几个因素可能更为重要。首先且最重要的是，选择的处理器必须市场上具有很大的存在感，即处理器的装机基数。原因非常简单。软件开发的成本最好由非常庞大的客户群体来证明。在市场存在感相对于通用微处理器可忽略不计的传统并行计算系统上，运行这些应用程序的应用程序的市场基础将不会很大。这一直是传统并行计算系统的一个主要问题。只有一些由政府和大公司资助的精英应用程序已成功开发在这些传统的并行计算系统上。而众多线程的GPU改变了这一点。由于它们在PC市场上的普及，GPU已经以数亿计的销量出售。几乎所有的台式机 and 高端笔记本电脑都配备有GPU。迄今为止，已经有超过10亿个支持CUDA的GPU在使用中。这样庞大的市场存在感使得这些GPU对应用程序开发者来说经济上具有吸引力。

另一个重要的决策因素是实际形式因素和易于获取性。直到2006年，并行软件应用程序运行在数据中心服务器或部门集群上。但这种执行环境往往会限制这些应用程序的使用。例如，在医学成像等应用中，基于64节点集群机器发表论文是可以接受的。但是实际的临床应用程序，如磁共振成像（MRI）机，通常基于PC和专用硬件加速器的某种组合。简单的原因是制造商，如GE和Siemens，在临床环境中无法销售需要机架式计算机服务器的MRI，而在学术部门环境中这是很常见的。事实上，美国国家卫生研究院（NIH）曾拒绝资助并行编程项目一段时间，因为他们认为并行软件的影响将受到限制，因为巨大的基于集群的机器在临床环境中无法使用。今天，许多公司使用GPU出货MRI产品，NIH资助使用GPU计算的研究。直到2006年，图形芯片非常难以使用，因为程序员必须使用图形API（应用程序编程接口）的等效功能来访问处理单元，这意味着需要使用OpenGL或Direct3D技术来编程这些芯片。简而言之，计算必须被表达为一个以某种方式绘制像素的函数，以便在这些早期GPU上执行。这种技术被称为GPGPU，即使用GPU进行通用目的编程。即使使用更高级的编程环境，底层代码仍然需要适应于绘制像素的API。这些API限制了实际可以为早期GPU编写的应用程序的类型。因此，GPGPU没有成为一种广泛的编程现象。尽管如此，这项技术足够令人兴奋，以激发一些英雄般的努力和出色的

研究结果。一切都在2007年改变，随着CUDA (NVIDIA, 2007) 的发布。CUDA不仅代表软件更改，还添加了芯片上的附加硬件。NVIDIA实际上投入了硅片面积，以促进并行编程的便利性。在用于并行计算的G80及其后续芯片中，GPGPU程序根本不再通过图形界面。相反，硅芯片上的新通用并行编程接口为CUDA程序服务。通用编程接口极大地扩展了可以轻松为GPU开发的应用程序的类型。所有其他软件层面也都进行了重新设计，以便程序员可以使用熟悉的C/C++编程工具。虽然GPU是异构并行计算中的重要计算设备类别，但还有其他重要类型的计算设备被用作异构计算系统中的加速器。例如，可编程门阵列已被广泛用于加速网络应用。本书中介绍的使用GPU作为学习工具的技术也适用于这些加速器的编程任务。

1.2 为什么需要更高速度或并行性？

正如我们在1.1节中所述，大规模并行编程的主要动机是让应用程序在未来硬件世代中继续享受速度提升。正如我们将在并行模式、高级模式 and 应用程序 (第二部分和第三部分，第7章至第19章) 的章节中讨论的那样，当一个应用程序适合并行执行时，在GPU上的良好实现可以使其速度提高超过100倍，相较于在单个CPU核上的顺序执行。如果应用程序包含我们所称的“数据并行性”，通常只需几小时的工作就可以实现 10^3 倍的加速。

有人可能会问，为什么应用程序会继续要求提高速度。我们今天拥有的许多应用程序似乎已经运行得足够快了。尽管当今世界存在着大量计算应用程序，但未来许多令人兴奋的大众市场应用程序实际上是我们以前认为是超级计算应用程序或超级应用程序的应用程序。例如，生物研究界越来越多地涉足分子层面。显微镜，可以说是分子生物学中最重要的仪器，过去依赖于光学或电子设备。然而，我们使用这些仪器进行分子层面的观察存在一些限制。通过将计算模型纳入传统仪器设定的边界条件来模拟基础分子活动，可以有效解决这些限制。通过模拟，我们可以测量比仅使用传统仪器想象得到的更多细节并测试更多假设。从长远来看，这些模拟将继续受益于计算速度的提高，以便建模的生物系统的大小和模拟的反应时间在可接受的响应时间内。

对于视频和音频编码与处理等应用程序，考虑一下我们对数字高清 (HD) 电视的满意度与旧的NTSC电视相比。一旦我们在高清电视上体验到画面的细节水平，就很难回到旧技术。但考虑到为那个高清电视所需的所有处理。这是一个高度并行的过程，就像三维 (3D) 成像和可视化一样。将来，新功能，如视图合成和低分辨率视频的高分辨率显示，将需要更多电视的计算能力。在消费者层面，我们将开始看到越来越多的视频和图像处理应用程序，改善图片和视频的焦点、照明和其他关键方面。

更多计算速度带来的好处之一是更好的用户界面。智能手机用户现在通过高分辨率触摸屏享受到与大屏电视媲美的自然界面。毫无疑问，这些设备的未来版本将整合传感器和具有3D透视的显示器，结合虚拟和物理空间信息的应用，以提高可用性，并需要更多计算速度。

类似的发展也正在消费电子游戏领域进行。过去，在游戏中驾驶汽车只是一组预先安排的场景。如果你的车撞到障碍物，你的车辆的路线不会改变；只有游戏得分会改变。你的车轮不会弯曲或受损，即使你失去一个轮子，驾驶起来也不会更加困难。随着计算速度的提高，游戏可以基于动态模拟而不是预先安排的场景。我们可以期望在未来体验到更多这些真实效果。事故将损坏你的车轮，你的在线驾驶体验将更加真实。准确模拟物理现象的能力已经激发了数字孪生的概念，其中物理对象在模拟空间中具有准确的模型，以便可以以更低的成本进行应力测试和退化预测。模拟物理效应的逼真建模已知需要大量计算能力。

通过大幅提高计算吞吐量实现的新应用的一个重要例子是基于人工神经网络的深度学习。尽管自上世纪70年代以来神经网络一直在积极研究，但它们在应用中效果不佳，因为训练这些网络需要太多标记数据和太多计算资源。互联网的兴起提供了大量标记图片，而GPU计算吞吐量的提升则带来了大量计算资源。因此，自2012年以来，基于神经网络的应用在计算机视觉和自然语言处理领域迅速得到采用。这种采用已经彻底改变了计算机视觉和自然语言处理应用，并促使了自动驾驶汽车和家庭助手设备的快速发展。

我们提到的所有新应用都涉及以不同的方式和在不同的层次模拟和/或表示一个物理且并发的世界，处理大量数据。在这么大量的数据下，许多计算可以在数据的不同部分并行进行，尽管它们最终需要在某一点进行协调。在大多数情况下，有效地管理数据传递对于并行应用程序可实现的速度具有重大影响。虽然这方面的技术通常

为那些每天与这类应用程序一起工作的专业人士所熟知，但绝大多数应用程序开发人员可以从更直观的理解和对这些技术的实际工作知识中受益。

我们的目标是以一种直观的方式向应用程序开发人员呈现数据管理技术，这些开发人员的正规教育可能不是计算机科学或计算机工程。我们还旨在提供许多实际的代码示例和实践练习，帮助读者获得实际的工作知识，这需要一个能够促进并行实现并支持适当管理数据传递的实际编程模型。CUDA提供了这样一种编程模型，并得到了大量开发者社区的充分测试。

1.3 加速实际应用程序

我们能从并行化应用程序中期望多大的加速？计算系统 A 上执行应用程序与计算系统 B 上执行相同应用程序所用时间的比率定义了应用程序的加速度。例如，如果一个应用程序在系统 A 中执行需要 10 秒，而在系统 B 中执行需要 200 秒，那么系统 A 对系统 B 的执行速度就是 $200/10=20$ ，称为 20 倍的速度提升。

并行计算系统相对于串行计算系统可以实现的加速度取决于可以并行化的应用程序部分。例如，如果在可以并行化的部分花费的时间百分比为 30%，那么并行部分的 100倍 速度提升将最多减少应用程序的总执行时间 29.7%。也就是说，整个应用程序的加速度只约为 $1/(1-0.297)=1.423$ 。事实上，即使在可以并行化的部分可以实现无限的速度提升，也只能在执行时间上削减 30%，最多达到 1.433 倍的速度提升。通过并行执行可以实现的速度提升水平可能受到应用程序可并行化部分的严重限制，这被称为阿姆达尔定律（Amdahl, 2013）。

另一方面，如果 99% 的执行时间在可以并行化的部分，那么并行部分的 100倍 速度提升将应用程序执行时间降低到原始时间的 1.99%。这给整个应用程序带来了 50倍 的速度提升。因此，对于大规模并行处理器有效地加速其执行，应用程序的绝大多数执行都在并行部分非常重要。

研究人员已经在某些应用程序上实现了超过 100倍 的加速度。然而，这通常仅在算法经过大量优化和调整 after 实现，以便超过 99.9% 的应用程序工作在并行部分。

影响应用程序可实现加速度水平的另一个重要因素是从内存访问数据的速度以及向内存写入数据的速度。在实践中，应用程序的直接并行化通常会饱和内存（DRAM）带宽，导致只有约 10倍 的速度提升。关键在于想出如何绕过内存带宽限制，这涉及进行许多转换，以利用专用 GPU 芯片上的内存，从而大大减少对 DRAM 的访问次数。然而，必须进一步优化代码，以避免诸如有限的芯片内存容量等限制。本书的一个重要目标是帮助读者充分理解这些优化并掌握它们的使用技巧。

请记住，相对于单核 CPU 执行实现的速度提升水平也反映了 CPU 对应用程序的适用性。在一些应用程序中，CPU 的性能非常好，这使得使用 GPU 加速性能变得更加困难。大多数应用程序有可以由 CPU 更好地执行的部分。必须公平地给 CPU 一个执行的机会，并确保代码编写得让 GPU 与 CPU 执行相辅相成，充分发挥结合 CPU/GPU 系统的异构并行计算能力。截至目前，结合多核 CPU 和多核 GPU 的大众市场计算系统已将TFLOPS 级别的计算引入了笔记本电脑，并将PFLOPS计算引入了集群。

图1.2说明了典型应用程序的主要部分。实际应用程序的代码很大一部分往往是顺序的。这些顺序部分被描绘为桃子的“核”区域；试图将并行计算技术应用于这些部分就像咬到桃核一样，感觉不好！这些部分很难并行化。CPU在这些部分上往往表现得非常出色。好消息是，尽管这些部分可能占据代码的很大一部分，但它们往往只占用超级应用程序执行时间的一小部分。

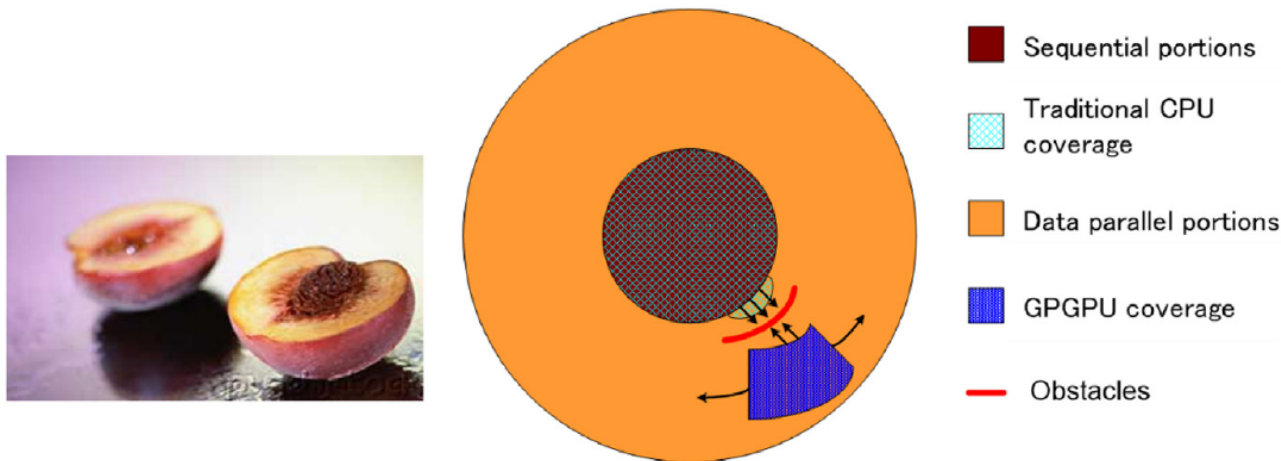


图1.2 顺序和并行应用部分的覆盖范围。顺序部分和传统（单核）CPU覆盖部分相互重叠。先前的GPGPU技术对数据并行部分的覆盖非常有限，因为它仅限于可以表达为绘制像素的计算。障碍物指的是难以扩展单核CPU以覆盖更多数据并行部分的功耗约束。

然后是我们所称的“桃肉”部分。这些部分很容易并行化，就像一些早期图形应用程序一样。异构计算系统中的并行编程可以大幅提高这些应用程序的速度。如图1.2所示，早期的GPGPU编程接口仅覆盖了桃肉部分的一小部分，类似于最令人兴奋的应用程序的一小部分。正如我们将看到的，CUDA编程接口旨在覆盖更大一部分令人兴奋的应用程序的桃肉。并行编程模型及其底层硬件仍在快速发展，以实现对应用程序更大部分的高效并行化。

1.4 并行编程中的挑战

并行编程为何如此困难？曾经有人说，如果你不关心性能，那么并行编程非常容易。你可以在一个小时内编写一个并程序。但如果你不关心性能，为什么还要费心编写并程序呢？本书解决了并行编程中实现高性能所面临的几个挑战。

首先，设计具有与顺序算法相同算法（计算）复杂性水平的并行算法可能是具有挑战性的。许多并行算法执行与它们的顺序对应物相同数量的工作。然而，有些并行算法执行的工作可能比它们的顺序对应物更多。事实上，有时它们可能执行的工作量如此之大，以至于在大型输入数据集上运行得更慢。这尤其是一个问题，因为快速处理大型输入数据集是进行并行编程的一个重要动机。例如，许多实际问题最自然地通过数学递归来描述。并行化这些问题通常需要以非直观的方式思考问题，并可能需要在执行过程中进行冗余工作。有一些重要的算法原语，如前缀和，可以促使将问题的顺序、递归公式转换为更并行的形式。我们将在第11章“前缀和（Scan）”中更正式地介绍工作效率的概念，并说明在设计与顺序对应物具有相同计算复杂性水平的并行算法中所涉及的方法和权衡，使用重要的并行模式，如前缀和。

其次，许多应用程序的执行速度受到内存访问延迟和/或吞吐量的限制。我们将这些应用程序称为内存受限应用程序；相比之下，计算受限应用程序受到每字节数据执行的指令数量的限制。在内存受限应用程序中实现高性能的并行执行通常需要改进内存访问速度的方法。我们将在第5章“内存体系结构和数据局部性”和第6章“性能考虑”中介绍内存访问的优化技术，并在介绍并行模式 and 应用程序的若干章节中应用这些技术。

第三，与其顺序对应物相比，并程序的执行速度通常更加敏感于输入数据的特性。许多实际应用程序需要处理具有广泛变化特性的输入，例如不规则或不可预测的数据大小和不均匀的数据分布。这些大小和分布的变化可能导致将不均匀的工作分配给并行线程，从而显著降低并行执行的有效性。并程序的性能有时可能会随着这些特性的变化而戏剧性地变化。我们将在介绍并行模式 and 应用程序的章节中介绍用于规范化数据分布和/或动态调整线程数量以解决这些挑战的技术。

第四，一些应用程序可以在几乎不涉及不同线程之间协作的情况下并行化。这些应用程序通常被称为“易并行”。其他应用程序则要求线程之间进行协作，这需要使用同步操作，如屏障或原子操作。这些同步操作对应用程序施加了开销，因为线程通常会发现自己在等待其他线程而不是执行有用的工作。我们将在整本书中讨论减少这种同步开销的各种策略。

幸运的是，大多数这些挑战已经得到研究人员的解决。在不同应用领域之间存在共同的模式，允许我们将在一个领域中推导出的解决方案应用到其他领域的挑战中。这是为什么我们将在重要的并行计算模式和应用程序的上下文中呈现解决这些挑战的关键技术的主要原因。

1.5 相关的并行编程接口

在过去的几十年中，提出了许多并行编程语言和模型（Mattson等，2004）。最广泛使用的是用于共享内存多处理器系统的OpenMP（Open, 2005）和用于可伸缩集群计算的消息传递接口（MPI）（MPI, 2009）。两者都已成为主要计算机供应商支持的标准化编程接口。

OpenMP实现包括编译器和运行时。程序员通过指定关于循环的指令（commands）和编译器的提示（hints）向OpenMP编译器提供信息。使用这些指令和提示，OpenMP编译器生成并行代码。运行时系统通过管理并行线程和资源来支持并行代码的执行。OpenMP最初是为CPU执行而设计的，并已扩展以支持GPU执行。

OpenMP的主要优势在于，它提供了编译器自动化和运行时支持，以使程序员从并行编程中抽象出许多细节。这种自动化和抽象有助于使应用代码在由不同供应商生产的系统以及同一供应商的不同系统世代之间更具可移植性。我们将这种属性称为性能可移植性。然而，在OpenMP中进行有效编程仍然需要程序员理解所涉及的所有详细并行编程概念。因为CUDA给程序员提供了对这些并行编程细节的显式控制，所以即使是希望将OpenMP作为其主要编程接口的人，它也是一个很好的学习工具。此外，根据我们的经验，OpenMP编译器仍在不断发展和改进。许多程序员可能需要在OpenMP编译器存在不足的部分使用CUDA风格的接口。

另一方面，MPI是一个计算节点在集群中不共享内存的编程接口（MPI, 2009）。所有数据共享和交互都必须通过显式消息传递来完成。MPI在高性能计算（HPC）中被广泛使用。在MPI中编写的应用程序已经成功在具有超过100,000个节点的集群计算系统上运行。今天，许多HPC集群使用异构的CPU/GPU节点。将应用程序移植到MPI中所需的工作量可能会相当大，这是由于计算节点之间缺乏共享内存。程序员需要进行领域分解，将输入和输出数据分区到各个节点。基于领域分解，程序员还需要调用消息发送和接收函数来管理节点之间的数据交换。相比之下，CUDA为GPU中的并行执行提供了共享内存以解决这一困难。虽然CUDA是与每个节点有效通信的接口，但大多数应用程序开发人员需要使用MPI在集群级别进行编程。此外，通过诸如NVIDIA Collective Communications Library（NCCL）的API，CUDA对多GPU编程的支持也越来越多。因此，对于在现代计算集群中使用多GPU节点的并程序员来说，理解如何进行MPI/CUDA联合编程是非常重要的，这是在第20章“编程异构计算集群”中介绍的一个主题。

在2009年，包括苹果、英特尔、AMD/ATI和NVIDIA在内的几家主要行业参与者共同开发了一种标准化的编程模型，称为Open Compute Language（OpenCL）（The Khronos Group, 2009）。与CUDA类似，OpenCL编程模型定义了语言扩展和运行时API，以允许程序员管理大规模并行处理器中的并行性和数据传递。与CUDA相比，OpenCL更多地依赖于API，而不是语言扩展。这使得供应商可以快速调整其现有的编译器和工具以处理OpenCL程序。OpenCL是一个标准化的编程模型，使用OpenCL语言扩展和API支持的所有处理器上的应用程序可以在不修改的情况下正确运行。但是，为了在新处理器上实现高性能，可能需要修改应用程序。

熟悉OpenCL和CUDA的人 would 知道，在OpenCL和CUDA的关键概念和特性之间存在显著的相似性。也就是说，CUDA程序员可以在很小的努力下学习OpenCL编程。更重要的是，几乎在CUDA中学到的所有技术都可以轻松应用于OpenCL编程。

1.6 总体目标

我们的主要目标是教会你，读者，如何在大规模并行处理器上编写高性能的程序。因此，书中的大部分内容都致力于开发高性能并行代码的技术。我们的方法不需要很多硬件专业知识。尽管如此，你仍需要对并行硬件架构有良好的概念理解，以便能够推理你的代码的性能行为。因此，我们将专门介绍一些关键硬件架构特性的直观理解，并将很多篇幅用于开发高性能并行程序的技术。特别是，我们将专注于计算思维（Wing, 2006）技术，使你能够以适合在大规模并行处理器上高性能执行的方式思考问题。

在大多数处理器上进行高性能并行编程需要一些关于硬件如何工作的知识。构建工具和机器使程序员能够在没有这些知识的情况下开发高性能代码可能需要很多年的时间。即使有了这样的工具，我们怀疑那些具有硬件知识的程序员将能够比那些没有硬件知识的程序员更有效地使用这些工具。因此，我们专门在第4章“计算架构与调度”中介绍GPU架构的基础知识。我们还将更专业的架构概念作为高性能并行编程技术讨论的一部分来讨论。

我们的第二个目标是教授正确功能和可靠性的并行编程，这在并行计算中是一个微妙的问题。之前在并行系统上工作过的程序员知道，仅仅实现初始性能是不够的。挑战在于以一种可以调试代码并支持用户的方式来实现它。CUDA编程模型鼓励使用简单形式的屏障同步、内存一致性和原子性来管理并行性。此外，它提供了一系列强大的工具，使人们能够调试不仅是功能方面，还包括性能瓶颈。我们将展示，通过专注于数据并行性，可以在应用程序中实现高性能和高可靠性。

我们的第三个目标是通过探索并行编程的方法，使未来的硬件代际能够在新一代的机器上比今天的机器运行得更快，从而实现可扩展性。我们希望帮助你掌握并行编程，以便你的程序能够达到新一代机器的性能水平。实现这种可扩展性的关键是规范和本地化内存数据访问，以最小化对关键资源的消耗和更新数据结构时的冲突。因此，开发高性能并行代码的技术对于确保未来应用程序的可扩展性也是重要的。

要实现这些目标，将需要许多技术知识，因此我们将在本书中涵盖并行编程原则和模式（Mattson等，2004）的许多原则和模式。我们将不会单独教授这些原则和模式。我们将在并行化有用应用程序的上下文中教授它们。然而，我们不能涵盖所有内容，因此我们选择了最有用和经过验证的技术来进行详细介绍。事实上，当前版本在并行模式方面的章节数量大大增加。现在，我们准备快速概述本书的其余部分。

1.7 书籍组织结构

本书分为四个部分。第一部分涵盖了并行编程、数据并行性、GPU和性能优化的基本概念。这些基础章节为读者提供了成为GPU程序员所必需的基本知识和技能。第二部分涵盖了基本并行模式，第三部分涵盖了更高级的并行模式和应用。这两部分应用了第一部分学到的知识和技能，并在需要时介绍其他GPU架构特性和优化技术。最后一部分，第四部分，介绍了高级实践，以完成那些想要成为专业GPU程序员的读者的知识体系。

第一部分关于基本概念包括第2至6章。第2章“异构数据并行计算”介绍了数据并行性和CUDA C编程。该章节基于读者之前具有C编程经验这一事实。它首先将CUDA C介绍为对C的简单、小的扩展，支持异构CPU/GPU计算和广泛使用的单程序多数据并行编程模型。然后，它涵盖了参与（1）识别要并行化的应用程序部分，（2）隔离要由并行化代码使用的数据，使用API函数在并行计算设备上分配内存，（3）使用API函数将数据传输到并行计算设备，（4）将并行部分开发为将由并行线程执行的核心函数，（5）通过API函数调用启动核心函数以由并行线程执行，以及（6）最终通过API函数调用将数据传输回主处理器。我们使用一个矢量加法的运行示例来说明这些概念。虽然第2章的目标是教授足够的CUDA C编程模型概念，以便读者能够编写简单的并行CUDA C程序，但它涵盖了基于任何并行编程接口开发并行应用程序所需的几个基本技能。

第3章“多维网格和数据”更详细地介绍了CUDA的并行执行模型，特别是与使用多维线程组织处理多维数据有关的部分。它提供了足够的见解，以便读者能够使用CUDA C实现复杂的计算。

第4章“计算架构与调度”介绍了GPU架构，重点关注计算核心的组织方式以及线程如何被调度在这些核心上执行。讨论了各种架构考虑因素，以及它们对在GPU架构上执行的代码性能的影响。其中包括透明可扩展性、SIMD执行和控制分歧、多线程和延迟容忍、占用等概念，这些都在本章中定义和讨论。

第5章“内存架构和数据局部性”扩展了第4章“计算架构与调度”，讨论了GPU的内存架构。它还讨论了用于保存CUDA变量以管理数据传递和提高程序执行速度的特殊内存。我们介绍了分配和使用这些内存的CUDA语言特性。恰当地使用这些内存可以极大地提高数据访问吞吐量，并有助于缓解内存系统中的流量拥塞。

第6章“性能考虑”介绍了当前CUDA硬件中的几个重要性能考虑因素。特别是，它提供了有关线程执行和内存访问良好模式的更多细节。这些细节形成了程序员推理有关组织计算和数据的决策对其性能影响的概念基础。本章以一个常见的优化策略清单结尾，GPU程序员经常用它来优化各种并行模式和应用程序，在本书的后两部分中将使用这个清单。

第二部分关于基本并行模式包括第7至12章。第7章“卷积”介绍了卷积，这是一种经常用于并行计算的模式，根植于数字信号处理和计算机视觉，并需要对数据访问局部性进行仔细管理。我们还使用这个模式来介绍现代GPU中的常量内存和缓存。第8章“模板”介绍了模板，这是一种类似于卷积但根植于求解微分方程的模式，具有特定的特征，为进一步优化数据访问局部性提供了独特的机会。我们还使用这个模式来介绍线程和数据的3D组织，并展示了第6章“性能考虑”中引入的针对线程粒度的优化。

第9章“并行直方图”涵盖了直方图，这是一种在统计数据分析和大数据集中的模式识别中广泛使用的模式。我们还使用这个模式来介绍原子操作作为协调对共享数据的并发更新的手段，以及私有化优化，它减少了这些操作的开销。第10章“归约和最小化分歧”介绍了归约树模式，它用于总结一组输入数据。我们还使用这个模式来演示控制分歧对性能的影响，并展示如何缓解这种影响的技术。第11章“前缀和（扫描）”介绍了前缀和，或称扫描，这是一种将固有的顺序计算转化为并行计算的重要模式。我们还使用这个模式来介绍并行算法中工作效率的概念。最后，第12章“合并”涵盖了并行合并，这是分而治之工作划分策略中广泛使用的模式。我们还利用这一章来介绍动态输入数据的识别和组织。

第三部分关于高级并行模式和应用在精神上类似于第二部分，但所涵盖的模式更为精细，并且通常包括更多的应用背景。因此，这些章节不太注重介绍新技术或特性，而更注重特定应用的考虑。对于每个应用，我们首先通过识别制定并行执行的基本结构的替代方式开始，并随后对每种替代方式的优缺点进行推理。然后，我们进行代码转换的步骤，以实现高性能。这些章节帮助读者将前几章的所有材料整合在一起，并在他们着手进行自己的应用开发项目时给予支持。

第三部分包括第13至19章。第13章“排序”介绍了两种形式的并行排序：基数排序和归并排序。这种高级模式利用了在前几章中涵盖的更基本的模式，特别是前缀和和并行合并。第14章“稀疏矩阵计算”介绍了稀疏矩阵计算，这在处理非常大的数据集时被广泛使用。该章向读者介绍了为了更有效地进行并行访问而重新排列数据的概念：数据压缩、填充、排序、转置和正则化。第15章“图遍历”介绍了图算法以及在GPU编程中如何高效实现图搜索。为图算法提供了许多不同的并行化策略，并讨论了图结构对最佳算法选择的影响。这些策略建立在更基本的模式之上，如直方图和合并。

第16章“深度学习”涵盖了深度学习，在GPU计算中变得极为重要。我们介绍了卷积神经网络的高效实现，并将更深入的讨论留给其他资料。卷积神经网络的高效实现利用了平铺等技术和卷积等模式。第17章“迭代磁共振成像重建”涵盖了非笛卡尔MRI重建，以及如何利用循环融合和分散收集转换等技术增强并行性并减少同步开销。第18章“电荷势图”涵盖了分子可视化和分析，通过应用从稀疏矩阵计算中学到的技术来处理不规则数据。

第19章“并行编程和计算思维”介绍了计算思维，即以更适于高性能计算的方式制定和解决计算问题的艺术。它通过讨论组织程序的计算任务，使其能够并行执行的概念来实现。我们首先讨论将抽象科学、问题特定概念转化为计算任务的平移过程，这是生成高质量应用软件（串行或并行）的重要第一步。然后，该章讨论并行算法结构及其对应用性能的影响，这是基于对CUDA性能调优经验的实践。尽管我们不涉及这些替代并行编程风格的实现细节，但我们期望读者能够通过在这本书中获得的基础来学习在其中任何一种中进行编程。我们还提供了一个高层次的案例研究，展示通过创造性的计算思维所能看到的机会。

第四部分关于高级实践包括第20至22章。第20章“编程异构计算集群”介绍了在异构集群上进行CUDA编程，其中每个计算节点都包含CPU和GPU。我们讨论了在CUDA和MPI并用以整合节点内计算和节点间计算以及由此产生的通信问题和实践。第21章“CUDA动态并行性”涵盖了动态并行性，即GPU根据数据或程序结构动态创建工作的能力，而不总是等待CPU这样做。第22章“高级实践和未来演进”列举了CUDA程序员需要了解的各种高级功能和实践。这些包括零拷贝内存、统一虚拟内存、多内核同时执行、函数调用、异常处理、调试、性能分析、双精度支持、可配置的缓存/刮擦板大小等。例如，CUDA的早期版本在CPU和GPU之间提供了有限的共享内存功能。程序员需要明确管理CPU和GPU之间的数据传输。然而，当前版本的CUDA支持统一虚拟内存和零拷贝内存等功能，实现了在CPU和GPU之间无缝共享数据。在这种支持下，CUDA程序员可以将变量和数据结构声明为在CPU和GPU之间共享。运行时硬件和软件保持一致性，并在需要时自动执行优化的数据传输操作。这种支持显著降低了在重叠数据传输与计算和I/O活动的复杂性中涉及的编程复杂性。在教材的开头部分，我们使用显式数据传输的API，以便读者更好地了解底层发生的情况。在第22章“高级实践和未来演进”中，我们稍后介绍了统一虚拟内存和零拷贝内存。

尽管本书各章基于CUDA，但它们帮助读者建立了通用并行编程的基础。我们相信人们通过具体的例子学习时能够更好地理解。也就是说，我们必须首先在特定的编程模型上学习概念，这为我们在将知识推广到其他编程模型时提供了坚实的基础。在这个过程中，我们可以利用CUDA示例的具体经验。与CUDA的深入经验还使我们能够获得成熟度，这将帮助我们学习甚至可能与CUDA模型无关的概念。第23章“结论和展望”提供了总结性的评论和对大规模并行编程未来的展望。我们首先回顾我们的目标，并总结各章如何共同助力实现这些目标。然后，我们以一个预测结束，即在未来十年中，大规模并行计算的快速进展将使其成为最激动人心的领域之一。